

Creating Threads and Understanding the Thread Lifecycle in Java

Coder Army

A thread represents an independent path of execution inside a Java program.

Java is an object-oriented language, and the `Thread` API is represented through objects. However, it is not technically correct to say that everything in Java is an object because primitive values such as `int`, `boolean`, and `double` are not objects.

The `Thread` class is itself a Java class:

```
public class Thread extends Object implements Runnable
```

A `Thread` object represents and controls a thread of execution.

This gives us two related but different ideas:

```
Thread object
→ Java object used to configure and control a thread

Thread of execution
→ independent sequence of instructions executed by the JVM
```

Creating a `Thread` object does not immediately start a new thread of execution. The new thread starts only when `start()` is called.

Ways to Create a Thread

The two foundational ways to create a platform thread using the `Thread` API are:

1. Extend the `Thread` class
2. Implement the `Runnable` interface

Modern Java also provides higher-level mechanisms such as:

- Executor services
- Thread pools
- `Callable` and `Future`
- Virtual threads
- Structured concurrency APIs

However, understanding `Thread` and `Runnable` first is important because the higher-level APIs build on the same separation between a task and its execution.

1. Creating a Thread by Extending `Thread`

Basic Idea

Create a class that extends `Thread` and override its `run()` method.

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

Create and start the thread:

```
public class Main {  
  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
  
        t1.start();  
    }  
}
```

```
}  
}
```

At this line:

```
MyThread t1 = new MyThread();
```

only a Java object has been created.

The thread has not started yet.

At this line:

```
t1.start();
```

the thread is scheduled to begin execution, and the new thread eventually executes:

```
run();
```

Conceptual Flow

```
new MyThread()  
  ↓  
Thread object is created  
  ↓  
State = NEW  
  ↓  
t1.start()  
  ↓  
JVM schedules a new thread of execution  
  ↓  
New thread receives its own execution context  
  ↓  
New thread executes run()
```

For a traditional platform thread, the JVM works with the operating system to schedule the thread. The exact native implementation is controlled by the JVM and operating system.

What Does a Thread Receive?

Each running thread has its own execution-related state, including:

```
Program counter
Java stack
Stack frames
Local variables
Current execution position
```

Threads in the same process normally share:

```
Heap objects
Static variables
Class metadata
Open process resources
```

This combination is what makes threads useful but also creates concurrency problems:

```
Separate execution stacks
+
shared heap data
```

Complete Example

```
class MyThread extends Thread {

    @Override
    public void run() {
        System.out.println(
            "Running in: "
            + Thread.currentThread().getName()
        );
    }
}
```

```

    }
}

public class Main {

    public static void main(String[] args) {
        MyThread worker = new MyThread();

        worker.setName("Worker-1");
        worker.start();

        System.out.println(
            "Main is running in: "
            + Thread.currentThread().getName()
        );
    }
}

```

The order of the two output lines is not guaranteed because the main thread and worker thread can execute concurrently.

2. Creating a Thread Using **Runnable**

Basic Idea

Instead of extending **Thread**, create a class that implements **Runnable**.

```

class MyRunnable implements Runnable {

    @Override
    public void run() {
        System.out.println(
            "Thread is running using Runnable"
        );
    }
}

```

```
}  
}
```

Pass the task to a `Thread` object:

```
public class Main {  
  
    public static void main(String[] args) {  
        MyRunnable task = new MyRunnable();  
  
        Thread t1 = new Thread(task);  
  
        t1.start();  
    }  
}
```

Internal Flow

MyRunnable object
→ represents the task

Thread object
→ represents the execution mechanism

Conceptually:

```
Thread t1 = new Thread(task)  
    ↓  
Thread stores the Runnable task  
    ↓  
t1.start()  
    ↓  
new thread begins execution  
    ↓  
Thread.run()  
    ↓  
task.run()
```

The default implementation of `Thread.run()` executes the `Runnable` task supplied to the thread.

Thread vs `Runnable`

3. Core Difference

The most useful mental model is:

```
Runnable
→ what work should be performed

Thread
→ which thread executes that work
```

Example:

```
Runnable task = () -> {
    System.out.println("Processing order");
};

Thread worker = new Thread(task);
worker.start();
```

Here:

```
Task      → process the order
Thread    → execute the task concurrently
```

4. `Thread` Implements `Runnable`

The `Thread` class itself implements the `Runnable` interface:

```
public class Thread
    extends Object
```

implements Runnable

This is why `Thread` has a `run()` method.

When extending `Thread`, the subclass overrides that inherited `run()` method.

When passing a `Runnable` to `Thread`, the thread's default `run()` method invokes the supplied task's `run()` method.

Why `Runnable` Is Usually Preferred

5. Separation of Concerns

Using `Runnable` separates the work from the mechanism that executes it.

```
Runnable → task definition
Thread   → execution and lifecycle
```

This makes the design easier to understand, test, reuse, and migrate to thread pools later.

6. Preserves Class Inheritance

Java classes support single class inheritance.

If a class extends `Thread`:

```
class ReportTask extends Thread {
}
```

it cannot extend another class.

Using `Runnable` avoids consuming the class's single inheritance opportunity:

```
class BaseReport {
}
```



```
class ReportTask
    extends BaseReport
    implements Runnable {

    @Override
    public void run() {
        System.out.println("Generating report");
    }
}
```

It is better to say:

Implementing `Runnable` preserves the ability to extend another class.

It does not mean that Java interfaces provide multiple class inheritance.

7. Reusability

The same task object can be executed by different threads:

```
Runnable task = new MyRunnable();

Thread t1 = new Thread(task, "Worker-1");
Thread t2 = new Thread(task, "Worker-2");

t1.start();
t2.start();
```

Both threads execute:

```
task.run();
```

However, this also means both threads share the same `task` object.

If the task contains mutable fields, those fields may require synchronization.

Example:

```
class CounterTask implements Runnable {
    private int count;

    @Override
    public void run() {
        count++;
    }
}
```

Passing the same `CounterTask` object to multiple threads means `count` is shared.

8. Works Naturally with Higher-Level APIs

The task-oriented design of `Runnable` works with executor services:

```
ExecutorService executor =
    Executors.newFixedThreadPool(2);

executor.submit(task);
executor.submit(task);

executor.shutdown();
```

The task does not need to know which particular thread executes it.

This separation is a major reason `Runnable` is generally preferred over extending `Thread`.

Comparison: `Thread` vs `Runnable`

Point	Extending <code>Thread</code>	Implementing <code>Runnable</code>
Main role	Combines task and thread	Defines only the task
Inheritance	Uses the class's single inheritance	Class can still extend another class

Point	Extending <code>Thread</code>	Implementing <code>Runnable</code>
Reusability	Task is closely tied to thread object	Same task can be submitted multiple times
Separation of concerns	Lower	Better
Works with executors	Not the natural design	Designed for task-based execution
Preferred for normal application code	Less common	Usually preferred

Using a Lambda Expression

9. `Runnable` Is a Functional Interface

`Runnable` contains one abstract method:

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

Therefore, it can be represented using a lambda expression.

Traditional form:

```
Runnable task = new Runnable() {

    @Override
    public void run() {
        System.out.println("Hello");
    }

};
```

Lambda form:

```
Runnable task = () -> {  
    System.out.println("Hello");  
};
```

Pass it to a thread:

```
Thread worker = new Thread(task);  
worker.start();
```

A shorter version is:

```
Thread worker = new Thread(  
    () -> System.out.println("Hello")  
);  
  
worker.start();
```

10. Lambda with Multiple Statements

```
Thread worker = new Thread(() -> {  
    System.out.println(  
        "Started: "  
        + Thread.currentThread().getName()  
    );  
  
    System.out.println("Performing task");  
  
    System.out.println("Task completed");  
}, "Worker-1");  
  
worker.start();
```

Using One Task with Multiple Threads

```
class MyRunnable implements Runnable {

    @Override
    public void run() {
        System.out.println(
            "Running in thread: "
            + Thread.currentThread().getName()
        );
    }
}
```

```
public class Main {

    public static void main(String[] args) {
        MyRunnable task = new MyRunnable();

        Thread t1 = new Thread(task, "Worker-1");
        Thread t2 = new Thread(task, "Worker-2");

        t1.start();
        t2.start();
    }
}
```

A possible output is:

```
Running in thread: Worker-2
Running in thread: Worker-1
```

Another execution may print the opposite order.

Thread Names and IDs

11. Every Thread Has an Identity

Threads have properties that help identify and inspect them, including:

```
Name
Thread ID
State
Priority
Daemon status
Interrupted status
```

12. Getting the Current Thread

Use:

```
Thread.currentThread();
```

Example:

```
Thread current = Thread.currentThread();

System.out.println(current.getName());
```

Inside `main()`, the current thread is normally named:

```
main
```

13. Getting a Thread Name

```
String name =
    Thread.currentThread().getName();
```

```
System.out.println(name);
```

14. Setting a Thread Name

Using `setName()`:

```
Thread worker = new Thread(() -> {
    System.out.println(
        Thread.currentThread().getName()
    );
});

worker.setName("Worker-Thread");
worker.start();
```

Using the constructor is usually cleaner:

```
Thread worker = new Thread(() -> {
    System.out.println(
        Thread.currentThread().getName()
    );
}, "Worker-Thread");

worker.start();
```

15. Getting a Thread ID

In modern Java, use:

```
long id =
    Thread.currentThread().threadId();

System.out.println("ID: " + id);
```

Older code commonly uses:

```
Thread.currentThread().getId();
```

However, `getId()` has been deprecated since Java 19. Modern code should use:

```
threadId();
```

A thread ID is useful for identification and debugging, but program logic should not normally depend on a particular numeric ID.

16. Why Names and IDs Matter

They help with:

- Debugging
- Logging
- Thread dumps
- Tracking execution
- Diagnosing deadlocks
- Understanding concurrent output
- Production monitoring

A meaningful name is often more useful than a numeric ID:

```
Thread-8
```

versus:

```
payment-worker-2
```

start() VS **run()**

17. Calling `run()` Directly

```
Thread worker = new Thread(() -> {
    System.out.println(
        "Current thread: "
        + Thread.currentThread().getName()
    );
}, "Worker-1");

worker.run();
```

Output:

```
Current thread: main
```

No new thread is created.

The `run()` method behaves like a normal method call and executes on the current thread.

Conceptually:

```
main thread calls worker.run()
→ main thread enters run()
→ main thread executes the task
→ main thread returns
```

18. Calling `start()`

```
worker.start();
```

Now the thread is scheduled independently.

Conceptually:

```
main thread calls worker.start()
→ JVM starts a new thread of execution
→ new thread invokes run()
→ main and worker may execute concurrently
```

Possible output:

```
Current thread: Worker-1
```

19. Comparison

<code>run()</code>	<code>start()</code>
Normal method call	Starts independent thread execution
Runs on calling thread	<code>run()</code> executes on the new thread
No new execution stack	New thread has its own execution stack
Can be invoked as a method more than once	A thread object can be started only once
No concurrent execution by itself	Enables concurrent execution

20. Demonstration Program

```
public class Main {

    public static void main(String[] args) {
        Runnable task = () -> {
            System.out.println(
                "Executed by: "
                + Thread.currentThread().getName
            );
        };

        Thread first = new Thread(task, "Worker-run");
```

```
Thread second = new Thread(task, "Worker-start");

first.run();
second.start();
}
}
```

Possible output:

```
Executed by: main
Executed by: Worker-start
```

Can a Thread Be Started Twice?

21. One **Thread** Object Can Be Started Only Once

```
Thread worker = new Thread(() -> {
    System.out.println("Running");
});

worker.start();
worker.start();
```

The second call throws:

```
java.lang.IllegalThreadStateException
```

A thread cannot be started again after it has:

- Already been started
- Completed normally
- Terminated because of an exception

22. Why Restarting Is Not Allowed

A `Thread` object represents one lifecycle:

```
NEW
→ started
→ active states
→ TERMINATED
```

After termination, that lifecycle is complete.

To perform the task again, create a new `Thread` object:

```
Runnable task = () -> {
    System.out.println("Running");
};

Thread first = new Thread(task);
first.start();

first.join();

Thread second = new Thread(task);
second.start();
```

The task can be reused, but the terminated `Thread` object cannot be restarted.

Thread Execution Order

23. Execution Order Is Not Guaranteed

Consider:

```
Thread t1 = new Thread(
    () -> System.out.println("Thread-1")
);
```

```
Thread t2 = new Thread(
    () -> System.out.println("Thread-2")
);

t1.start();
t2.start();
```

The order of the `start()` calls does not guarantee the order of execution.

Possible output:

```
Thread-1
Thread-2
```

Another possible output:

```
Thread-2
Thread-1
```

Calling `t1.start()` first means only that `t1` is made eligible for scheduling first. It does not guarantee that `t1` completes or even prints before `t2`.

24. Even and Odd Number Example

```
public class Main {

    public static void main(String[] args) {
        Thread evenThread = new Thread(() -> {
            for (int i = 2; i <= 10; i += 2) {
                System.out.println("Even: " + i);
            }
        }, "Even-Thread");

        Thread oddThread = new Thread(() -> {
```

```
        for (int i = 1; i <= 9; i += 2) {
            System.out.println("Odd: " + i);
        }
    }, "Odd-Thread");

    evenThread.start();
    oddThread.start();
}
}
```

A possible output is:

```
Even: 2
Odd: 1
Odd: 3
Even: 4
Even: 6
Odd: 5
```

The output may change every time the program runs.

25. What Influences Execution Order?

Execution order may be influenced by:

- Operating-system scheduling
- JVM scheduling decisions
- Number of processor cores
- Current system load
- Other running processes
- Thread priority
- Blocking operations
- Lock contention
- Input/output operations

- Timing of thread creation

Application code should not depend on accidental scheduling order.

26. Non-Determinism

Multithreaded execution is often non-deterministic.

```
Same program
+
same input
+
different scheduling
→ potentially different output order
```

Non-determinism does not automatically mean the program is incorrect.

It becomes a correctness problem when the final result depends on an uncontrolled execution order.

Use proper coordination mechanisms when an order is required:

- `join()`
 - Locks
 - `wait()` and `notifyAll()`
 - Latches
 - Barriers
 - Semaphores
 - Blocking queues
 - Executors and futures
-

Thread Lifecycle

A Java thread passes through a defined set of states.

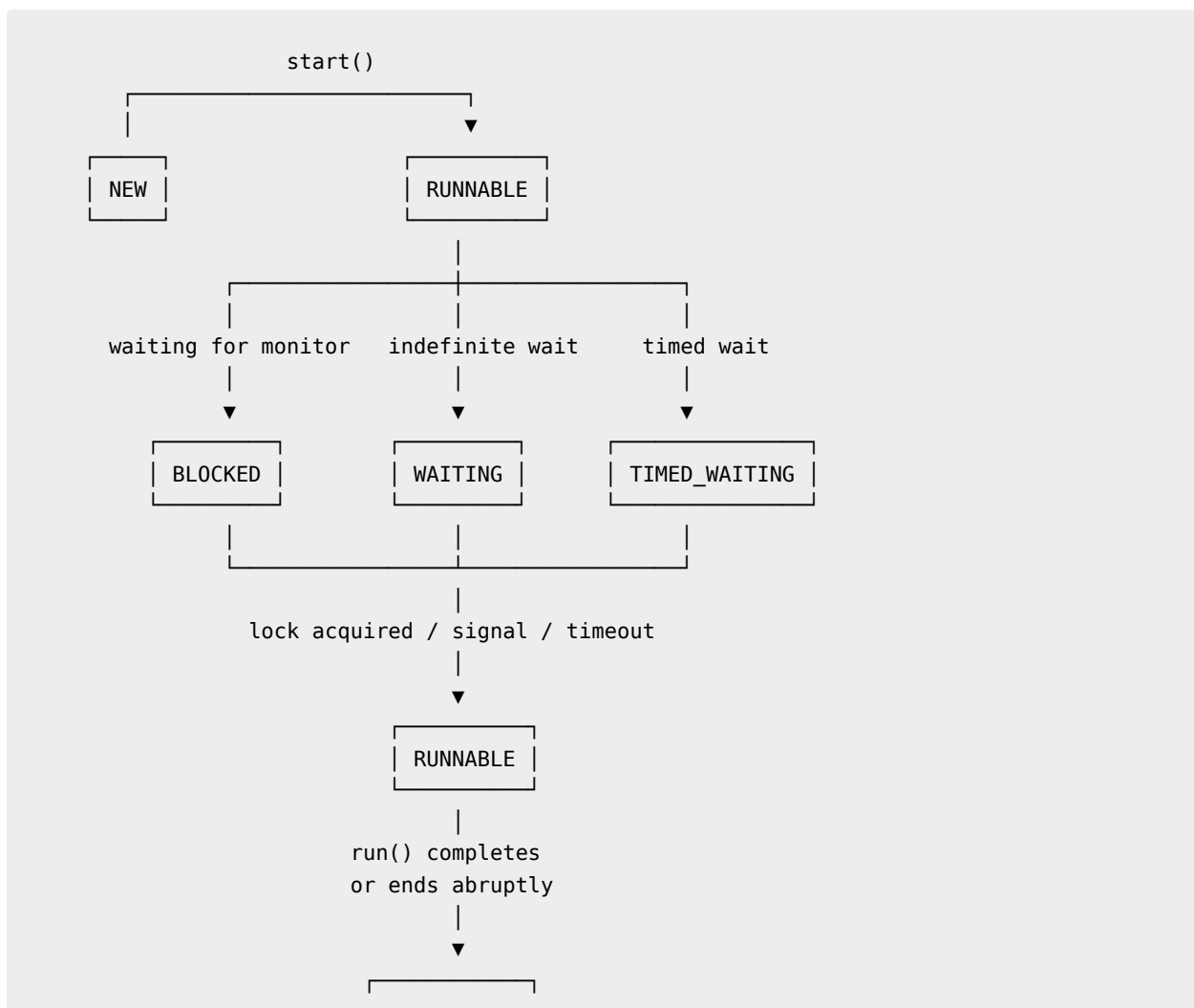
Java represents these states using:

Thread.State

The six states are:

```
NEW
RUNNABLE
BLOCKED
WAITING
TIMED_WAITING
TERMINATED
```

Simplified Thread Lifecycle Diagram



TERMINATED

This diagram is simplified. A thread may move between `RUNNABLE`, `BLOCKED`, `WAITING`, and `TIMED_WAITING` many times before termination.

State 1: `NEW`

27. Meaning

A thread is in the `NEW` state after the `Thread` object has been created but before `start()` has been called.

```
Thread worker = new Thread(() -> {  
    System.out.println("Running");  
});
```

At this point:

```
Thread state = NEW
```

Check it using:

```
System.out.println(worker.getState());
```

Output:

```
NEW
```

28. What Exists in the `NEW` State?

The Java `Thread` object exists.

However, independent thread execution has not started.

Thread object exists
but
`run()` is not executing concurrently

Calling `run()` directly does not move the thread through its normal lifecycle because `start()` was not called.

State 2: **RUNNABLE**

29. Meaning

After `start()` is called, a thread normally enters the **RUNNABLE** state.

```
worker.start();
```

Transition:

```
NEW  
→ RUNNABLE
```

In Java, **RUNNABLE** includes both:

```
Ready to receive processor time  
+  
currently executing in the JVM
```

Java does not expose a separate public **RUNNING** state in `Thread.State`.

30. Why Ready and Running Are Combined

The operating system ultimately schedules processor time.

A thread may repeatedly move between:

Waiting for CPU
↔ executing on CPU

Java groups both situations under:

`Thread.State.RUNNABLE`

A thread reported as `RUNNABLE` is not necessarily executing at that exact instant.

State 3: **BLOCKED**

31. Meaning

A thread enters `BLOCKED` when it is waiting to acquire an intrinsic monitor required to enter or re-enter synchronized code.

Example:

```
class SharedResource {  
  
    synchronized void use() {  
        try {  
            Thread.sleep(3000);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
}
```

```
SharedResource resource = new SharedResource();  
  
Thread t1 = new Thread(resource::use);  
Thread t2 = new Thread(resource::use);
```

```
t1.start();
t2.start();
```

If `t1` owns the monitor and is inside `use()`, `t2` may enter:

BLOCKED

because it is waiting for the same monitor.

32. Mental Model

BLOCKED

→ "I need a monitor to enter synchronized code,
but another thread currently owns it."

When the monitor becomes available and the thread acquires it:

BLOCKED

→ RUNNABLE

33. Does a **BLOCKED** Thread Release Locks?

A thread in the **BLOCKED** state is waiting to acquire a particular monitor.

It has not acquired that requested monitor yet.

However, it does not automatically release other monitors it may already hold.

Therefore, the statement:

"A blocked thread releases all monitor locks"

is incorrect.

Thread states do not automatically release all locks.

State 4: **WAITING**

34. Meaning

A thread is in **WAITING** when it waits indefinitely for another thread to perform a specific action.

Common causes include:

```
object.wait();  
thread.join();  
LockSupport.park();
```

35. **WAITING** Through **Object.wait()**

```
synchronized (lock) {  
    lock.wait();  
}
```

The thread:

```
Releases lock's monitor  
→ enters lock's waiting set  
→ enters WAITING
```

It may later be awakened through:

```
lock.notify();  
lock.notifyAll();
```

After notification, it must reacquire that same monitor before returning from **wait()**.

36. **WAITING** Through **join()**

```
worker.join();
```

The current thread waits until `worker` terminates.

It does not wait for `notify()`.

Therefore, not every thread in `WAITING` resumes through `notify()` or `notifyAll()`.

The required event depends on the method:

```
Object.wait()
→ notification, interruption, or spurious wakeup

Thread.join()
→ target thread termination or interruption

LockSupport.park()
→ unpark, interruption, or permitted spurious return
```

37. Does `WAITING` Release All Locks?

No.

Lock behaviour depends on the operation that caused the state.

`Object.wait()`

```
synchronized (lock) {
    lock.wait();
}
```

This releases only the monitor of `lock`.

It does not automatically release other monitors held by the thread.

`Thread.join()`

Calling `join()` does not generally release unrelated monitors already held by the calling thread.

`LockSupport.park()`

Parking does not automatically release intrinsic monitors.

Therefore:

A thread state alone does not tell us whether locks were released. The method that caused the state determines the lock behaviour.

State 5: **TIMED_WAITING**

38. Meaning

A thread enters **TIMED_WAITING** when it waits for another action for a limited period.

Common causes include:

```
Thread.sleep(2000);  
object.wait(2000);  
thread.join(2000);  
LockSupport.parkNanos(...);  
LockSupport.parkUntil(...);
```

39. Mental Model

```
TIMED_WAITING  
→ "I am waiting for an event or timeout,  
but not indefinitely."
```

The thread can become runnable again because:

- The timeout expires
- The expected event occurs
- The thread is interrupted
- The relevant API permits another form of wake-up

40. Lock Behaviour in **TIMED_WAITING**

Different timed-waiting methods behave differently.

Thread.sleep(2000)

Does not release monitors

object.wait(2000)

Releases the monitor of object

The thread must reacquire it before returning from `wait()`.

thread.join(2000)

Does not generally release unrelated monitors
held by the calling thread

Therefore, this statement is incorrect:

"TIMED_WAITING never releases monitor locks."

The correct answer depends on which method caused the state.

State 6: **TERMINATED**

41. Meaning

A thread enters **TERMINATED** after it finishes execution.

This can happen because:

- `run()` completes normally
- `run()` exits with an uncaught exception

Example:

```
Thread worker = new Thread(() -> {
    System.out.println("Done");
});

worker.start();
worker.join();

System.out.println(worker.getState());
```

Output:

TERMINATED

42. A Terminated Thread Cannot Restart

```
worker.start();
```

after termination throws:

IllegalThreadStateException

Create a new `Thread` object to execute the task again.

Comparing **BLOCKED**, **WAITING**, and **TIMED_WAITING**

State	Main Reason	Typical Example	How It Continues
BLOCKED	Waiting to acquire an intrinsic monitor	Entering occupied synchronized block	Acquires monitor

State	Main Reason	Typical Example	How It Continues
<code>WAITING</code>	Waiting indefinitely for an event	<code>wait()</code> , <code>join()</code> , <code>park()</code>	Notification, termination, unpark, interruption, etc.
<code>TIMED_WAITING</code>	Waiting with a time limit	<code>sleep()</code> , timed <code>wait()</code> , timed <code>join()</code>	Event, interruption, or timeout

A useful mental model is:

```

BLOCKED
→ lock acquisition problem

WAITING
→ indefinite coordination wait

TIMED_WAITING
→ limited-duration coordination or delay

```

Lock-Release Comparison

Operation	Typical State	Releases Monitor?
Waiting to enter <code>synchronized</code>	<code>BLOCKED</code>	Requested monitor was not acquired; other held monitors are not automatically released
<code>Object.wait()</code>	<code>WAITING</code>	Releases only the monitor on which <code>wait()</code> is called
<code>Object.wait(timeout)</code>	<code>TIMED_WAITING</code>	Releases only the monitor on which <code>wait()</code> is called
<code>Thread.sleep()</code>	<code>TIMED_WAITING</code>	No
<code>Thread.join()</code>	<code>WAITING</code>	Does not generally release unrelated monitors held by caller
<code>Thread.join(timeout)</code>	<code>TIMED_WAITING</code>	Does not generally release unrelated monitors held by caller
<code>LockSupport.park()</code>	<code>WAITING</code>	Does not automatically release intrinsic monitors

Printing Thread States

43. Basic State Demonstration

```
public class Main {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Thread worker = new Thread(() -> {  
            System.out.println("Running...");  
        });  
  
        System.out.println(  
            "Before start: " + worker.getState()  
        );  
  
        worker.start();  
  
        System.out.println(  
            "After start: " + worker.getState()  
        );  
  
        worker.join();  
  
        System.out.println(  
            "After completion: " + worker.getState()  
        );  
    }  
}
```

A likely output is:

```
Before start: NEW  
After start: RUNNABLE
```

After completion: TERMINATED

However, the state immediately after `start()` is timing-sensitive.

If the task completes very quickly, the second line may already report:

TERMINATED

Thread state observations are snapshots and can change immediately after being read.

44. Reliable `TIMED_WAITING` Demonstration

```
public class Main {

    public static void main(String[] args)
        throws InterruptedException {

        Thread worker = new Thread(() -> {
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        worker.start();

        Thread.sleep(500);

        System.out.println(worker.getState());

        worker.join();
    }
}
```

```
}  
}
```

The worker will most likely be:

```
TIMED_WAITING
```

when inspected.

45. Demonstrating **BLOCKED**

```
public class Main {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Object lock = new Object();  
  
        Thread t1 = new Thread(() -> {  
            synchronized (lock) {  
                try {  
                    Thread.sleep(5000);  
                } catch (InterruptedException e) {  
                    Thread.currentThread().interrupt();  
                }  
            }  
        }, "Thread-1");  
  
        Thread t2 = new Thread(() -> {  
            synchronized (lock) {  
                System.out.println("Thread-2 entered");  
            }  
        }, "Thread-2");  
    }  
}
```

```
t1.start();

Thread.sleep(200);

t2.start();

Thread.sleep(200);

System.out.println(
    "Thread-2 state: " + t2.getState()
);

t1.join();
t2.join();
}
}
```

While `t1` owns `lock`, `t2` will most likely report:

BLOCKED

46. Demonstrating **WAITING**

```
public class Main {

    public static void main(String[] args)
        throws InterruptedException {

        Object lock = new Object();

        Thread worker = new Thread(() -> {
            synchronized (lock) {
                try {
                    lock.wait();
                }
            }
        });
```

```

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

worker.start();

Thread.sleep(200);

System.out.println(worker.getState());

synchronized (lock) {
    lock.notifyAll();
}

worker.join();
}
}

```

The worker will most likely report:

WAITING

before notification.

Important Lifecycle Rules

1. Creating a `Thread` object places it in `NEW`.
2. Calling `run()` directly does not start a new thread.
3. Calling `start()` schedules the thread to execute `run()`.
4. A thread object can be started only once.
5. Java's `RUNNABLE` state includes ready and actively executing situations.

6. `BLOCKED` specifically means waiting for an intrinsic monitor.
7. `WAITING` does not always mean waiting for `notify()`.
8. `TIMED_WAITING` can be caused by `sleep()`, timed `wait()`, timed `join()`, and parking methods.
9. `Object.wait()` releases only the monitor on which it is called.
10. `sleep()` does not release monitors.
11. A blocked or waiting thread does not automatically release all locks.
12. A thread enters `TERMINATED` when `run()` finishes normally or abruptly.
13. A terminated thread cannot be restarted.
14. `getState()` returns only a momentary snapshot.
15. Thread execution order should never be assumed from `start()` order.

Final Comparison: Extending `Thread` vs Implementing `Runnable`

Extending `Thread`

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println("Task");  
    }  
}  
  
new MyThread().start();
```

Implementing `Runnable`


```
class MyTask implements Runnable {  
  
    @Override  
    public void run() {  
        System.out.println("Task");  
    }  
}
```

```
Runnable task = new MyTask();  
new Thread(task).start();
```

Lambda

```
new Thread(  
    () -> System.out.println("Task")  
).start();
```

The common execution flow is:

```
Create Thread object  
→ call start()  
→ thread becomes eligible to run  
→ new thread executes run()  
→ run() completes  
→ thread terminates
```

Final Summary

```
Thread  
→ represents and controls a thread of execution  
  
Runnable  
→ represents a task with no return value  
  
Extending Thread
```

→ combines task and thread lifecycle

Implementing Runnable

→ separates task from execution

run()

→ normal method when called directly

start()

→ starts independent thread execution

Thread execution order

→ not guaranteed

Thread lifecycle

→ NEW

→ RUNNABLE

→ possibly BLOCKED / WAITING / TIMED_WAITING

→ RUNNABLE

→ TERMINATED

For basic learning, creating `Thread` objects directly makes the lifecycle visible. In production applications, tasks are generally submitted to executors or other higher-level concurrency APIs rather than manually creating a new platform thread for every task.